

Ok so OOP... honestly it took me a while to really get this when I first started coding last semester. I kept mixing up classes and objects it was super confusing at first. But basically think of a class like a blueprint. If you wanna build a house you don't draw a new blueprint every time right you just use the same one and build multiple houses from it. Objects are the actual houses.

So the four pillars thing:

**ENCAPSULATION** this one I actually understood from day 1. It's basically hiding stuff. Like when I use my phone I don't need to know HOW the camera app works inside, I just press a button and it works. Same idea in code you put data and functions together inside a class and mark sensitive stuff as private so other code can't just randomly mess with it. We did this in lab 3 with the BankAccount class. Sir marked my balance variable as public and deducted marks lol, lesson learned.

**ABSTRACTION** ok this is similar to encapsulation but apparently different?? Like it's more about simplifying for the person using your class. You give them clean methods and hide the complicated stuff inside. Honestly I still sometimes mix these two up. Maybe they're two sides of the same coin? I should ask sir.

**INHERITANCE** this one's my favourite actually. So if I make an Animal class with an eat() method, both Dog and Cat classes can just inherit from Animal and they automatically get eat(). No need to rewrite it. My friend Ravi warned me though that if you make inheritance chains too deep it becomes a nightmare to debug. I didn't believe him until I spent 3 hours on a bug last week that was exactly that problem.

**POLYMORPHISM** ok so this blew my mind a little. Same method name but different behaviour depending on which object you call it on. Dog.speak() says woof, Cat.speak() says meow, even though the method is called speak() in both. We had this question in the mid-sem and I got it wrong because I didn't realize you need to override the method in the subclass, not just inherit it. Makes sense now in hindsight.

Overall I think OOP is pretty powerful once it clicks. Not gonna lie it took a few weeks and some real debugging pain to really get it. Would've been nice to see more real-world examples in class though.